

EXPERIMENT-1

Name: K.Vaishali

Rollno: 2520030117

Section: 8

Experiment No. 1

Title: Implementation of Process Creation and Command Execution in Linux using `fork()`, `exec()`, and `wait()` System Calls.

Aim: To develop a C program that accepts a Linux command from the user, creates a child process using `fork()`, executes the command using the `exec()` system call, waits for the child process using `wait()`, and displays the Process IDs (PIDs) of both parent and child processes.

Objective:

- To understand process creation in Linux.
- To learn how `fork()` creates a child process.
- To execute Linux commands using the `exec()` family of system calls.
- To synchronize parent and child processes using `wait()`.
- To observe the Process IDs (PIDs) of parent and child processes.

Theory: Linux is a multitasking operating system that executes multiple processes simultaneously. Every running program in Linux is known as a process and is identified by a unique Process ID (PID).

The operating system provides several system calls to manage

processes: **1. `fork()`:**

- Creates a new child process.
- The child process is an exact copy of the parent.
- Returns:
 - 0 to the child process.
 - Child PID to the parent process.
 - -1 if creation fails.

2. **exec():**

The `exec()` family replaces the current process image with a new program. Common functions are:

- `execl()`
- `execv()`
- `execlp()`
- `execvp()`

In this experiment, the child process executes the user-entered Linux command. **3. `wait()`:**

The parent process waits until the child process finishes execution. This prevents zombie processes.

Algorithm:

1. Start the program.
2. Accept a Linux command from the user.
3. Create a child process using `fork()`.
4. If child process:
 - Execute the command using `execlp()` or `execvp()`.
5. If parent process:
 - Wait for the child process using `wait()`.
6. Display Parent PID and Child PID.
7. Stop.

System Calls Used:

System Call	Purpose
<code>fork()</code>	Creates a child process
<code>exec()</code>	Executes a Linux command

wait()	Waits for child process
getpid()	Returns current process ID

System Call	Purpose
getppid()	Returns parent process ID

Sample Input: ls

Sample Output:

Enter Linux Command: ls

Parent Process

Parent PID: 3046

Child PID: 3047

Child Process

Child PID: 3047

Parent PID: 3046

hello.text process process.c

Child process completed.

Observation:

- A child process was successfully created using the fork() system call.
- The child process executed the user-entered Linux command using the exec() system call.
- The parent process waited until the child completed execution using the wait() system call.
- Different Process IDs (PIDs) were displayed for both parent and child processes.
- The experiment demonstrated how Linux creates and manages

processes. **Result:**

The program was successfully executed. The Linux command entered by the user was executed through a child process. The parent process waited for the child process to complete execution, and the Process IDs of both parent and child were displayed successfully.

Part 2: Linux Commands Observation

Aim: To investigate the relationship between hardware resources and operating system services using Linux terminal commands.

Commands Used:

1. uname:

Purpose: Displays information about the Linux operating system.

Observation: Shows the kernel name and version of the operating system. **2. lscpu:**

Purpose: Displays CPU architecture and processor details.

Observation: Provides information about CPU model, cores, threads, cache size, and architecture.

3. lsblk:

Purpose: Lists storage devices connected to the system.

Observation: Displays hard disks, SSDs, partitions, and mounted storage devices. **4. ps:**

Purpose: Displays currently running processes.

Observation: Shows Process IDs (PIDs), terminal information, execution time, and running commands.

5. top:

Purpose: Monitors system performance in real time.

Observation: Displays CPU usage, memory usage, running processes, and system load dynamically.

Hardware and OS Relationship:

CPU: The operating system schedules CPU time among multiple processes using the CPU scheduler, ensuring efficient execution.

Memory: The operating system manages RAM allocation and deallocation, preventing conflicts between processes through virtual memory and memory protection.

Storage: The operating system manages files, directories, and storage devices using the file system, allowing users to store and retrieve data efficiently.

Input/Output Devices: The operating system communicates with hardware devices such as keyboards, mice, printers, and disks using device drivers.

Conclusion: The experiment successfully demonstrated Linux process creation, execution, and synchronization using `fork()`, `exec()`, and `wait()`. It also showed how Linux commands provide detailed information about CPU, memory, storage, and running processes, illustrating the operating system's role in managing hardware resources.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <string.h>

int main() {

    char command[100];

    printf("Enter Linux command: ");

    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0) {

        printf("Fork failed!\n");

        return 1;

    }

    if (pid == 0) {

        printf("\nChild Process\n");
```

```

printf("Child PID : %d\n", getpid());

printf("Parent PID: %d\n", getppid());

execlp(command, command, NULL);

printf("Command not found!\n");

exit(1);

}

else {

printf("\nParent Process\n");
printf("Parent PID: %d\n", getppid());

printf("Child PID : %d\n", pid);

wait(NULL);

printf("\nChild process completed.\n");

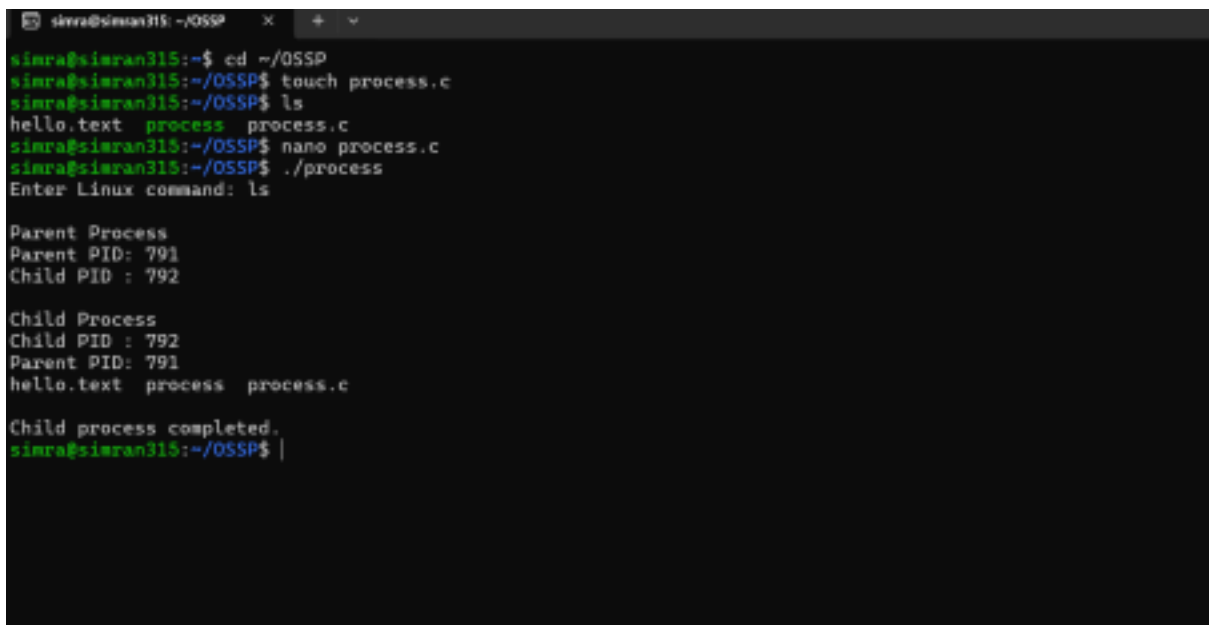
}

return 0;

}

```

OUTPUT:



```

simra@simran315: ~/OSSP
simra@simran315:~$ ed ~/OSSP
simra@simran315:~/OSSP$ touch process.c
simra@simran315:~/OSSP$ ls
hello.text  process  process.c
simra@simran315:~/OSSP$ nano process.c
simra@simran315:~/OSSP$ ./process
Enter Linux command: ls

Parent Process
Parent PID: 791
Child PID : 792

Child Process
Child PID : 792
Parent PID: 791
hello.text  process  process.c

Child process completed.
simra@simran315:~/OSSP$ |

```